**การเผยแพร่ข้อสอบ**

☐ อนุญาต ☒ ไม่อนุญาต

**เลขที่นั่งสอบ** _____

**วันที่สอบ**

27 ตุลาคม 2569

**เวลาสอบ**

13.00 - 16.00 (3 ชั่วโมง)

**กลุ่มนักศึกษา**

☒ ปกติ

☒ International

**ชุดข้อสอบ (ถ้ามี)** : 1

วิชา

# CPE334

## Software Engineering

**ชื่อ-นามสกุล :** _____ **รหัสนักศึกษา :** _____

**ภาควิชา/สาขาวิชา :** _____ **กลุ่ม :** _____

**คำชี้แจง**

1. ข้อสอบมี **3 ส่วน 12 ข้อ คะแนนเต็ม 100 คะแนน** จำนวน 32 หน้า (รวมหน้าปก)
   - ส่วนที่ 1 Requirements และ Security ข้อ 1-4 รวม 25 คะแนน
   - ส่วนที่ 2 Project Planning และ Execution ข้อ 5-7 รวม 25 คะแนน
   - ส่วนที่ 3 UML/UI และ Git Workflow ข้อ 8-12 รวม 50 คะแนน

   **ให้ทำคำตอบลงในข้อสอบ**
2. เขียนชื่อและรหัสนักศึกษาลงในข้อสอบทุกหน้า
3. ใช้ปากกาหมึกสีน้ำเงินหรือสีดำ หากพื้นที่คำตอบไม่พอ ให้เขียนต่อด้านหลังของหน้าข้อสอบข้อเดียวกัน
4. ไม่มีการชี้แจงหรือแก้ไขโจทย์ระหว่างสอบ หากโจทย์ไม่ชัดเจนหรือมีข้อสงสัย ให้ตั้งสมมติฐานที่สมเหตุสมผลและอธิบายเหตุผลประกอบคำตอบ

**เงื่อนไขการสอบ**

1. **ไม่อนุญาตให้นำเครื่องคำนวณเข้าห้องสอบ**
2. อนุญาตให้นำกระดาษสรุป A4 1 แผ่นเข้าห้องสอบ โดย**ไม่ต้องส่งกระดาษสรุปเนื้อหาพร้อมข้อสอบ**

* ข้อสอบได้ผ่านการพิจารณาจากคณะกรรมการประจำภาควิชาวิศวกรรมคอมพิวเตอร์แล้ว *

| **หัวหน้าภาควิชา** | **อาจารย์ผู้สอน / ผู้ประสานงานรายวิชา** |
|---|---|
| ..... | ..... |
| | เบอร์โทรศัพท์ : ..... |

# ส่วนที่ 1 (25 คะแนน) ข้อกำหนดและความมั่นคงปลอดภัย

**ข้อ 1 (8 คะแนน): วิเคราะห์ความต้องการของผู้มีส่วนได้ส่วนเสีย**

| | |
|---|---|
| A university is preparing a workshop registration system. Its coordinator gives the development team four requirement statements. No additional details have yet been agreed beyond those stated. | มหาวิทยาลัยกำลังเตรียมระบบลงทะเบียน workshop ผู้ประสานงานส่งข้อความความต้องการสี่ข้อให้ทีมพัฒนา แต่ทีมยังไม่ได้ยืนยันรายละเอียดอื่นนอกเหนือจากข้อความที่ให้ |

ให้วิเคราะห์แต่ละข้อแยกกัน โดยใช้หัวข้อต่อไปนี้

จำแนกประเภทเป็น functional requirement, non-functional requirement หรือ business/domain rule พร้อมเหตุผล อธิบายจุดประสงค์ ประเมินคุณภาพและจุดที่ยังไม่ชัด เสนอวิธีทดสอบหรือวัดพร้อมผลที่คาดหวัง แล้วปรับข้อความหรืออธิบายเหตุผลที่ควรคงไว้ ตัวเลขหรือเงื่อนไขที่เสนอเพิ่มต้องระบุว่ารอยืนยันกับผู้เกี่ยวข้อง

**1.1 (2 คะแนน) ดู workshop ที่เปิดรับลงทะเบียน**

| | |
|---|---|
| “The system shall let a signed-in student view all workshops that are open for registration.” | “ระบบต้องให้นักศึกษาที่เข้าสู่ระบบแล้วดู workshop ทั้งหมดที่เปิดรับลงทะเบียนได้” |

วิเคราะห์ข้อความนี้ตามหัวข้อของข้อ 1 ให้ครบ

**คำตอบ**

**ข้อ 1 (ต่อ): ใช้หัวข้อวิเคราะห์เดียวกับหน้าแรกของข้อ 1**

**1.2 (2 คะแนน) ความเร็วในการลงทะเบียน**

| | |
|---|---|
| “The registration process should be quick.” | “กระบวนการลงทะเบียนควรรวดเร็ว” |

อธิบายว่าข้อความนี้บอกอะไรและยังขาดข้อมูลใด เสนอวิธีตรวจและผลที่คาดหวังซึ่งสอดคล้องกับการตีความของคุณ หากเพิ่มเกณฑ์ขึ้นเอง ให้ระบุว่าเป็นข้อเสนอที่ยังต้องยืนยัน

**คำตอบ**

**1.3 (2 คะแนน) ข้อจำกัดตามนโยบายมหาวิทยาลัย**

| | |
|---|---|
| “University policy limits each student to three workshop registrations per semester.” | “ตามนโยบายมหาวิทยาลัย นักศึกษาแต่ละคนลงทะเบียน workshop ได้ไม่เกิน 3 รายการต่อภาคการศึกษา” |

วิเคราะห์ตามหัวข้อเดิม โดยไม่เปลี่ยนนโยบายที่ให้ และระบุข้อมูลที่ควรถามเพิ่มก่อนนำไปพัฒนาหรือทดสอบ

**คำตอบ**

**ข้อ 1 (ต่อ): วิเคราะห์ข้อความข้อสุดท้าย**

**1.4 (2 คะแนน) ยกเลิกการลงทะเบียนและดูรายชื่อ**

| | |
|---|---|
| “The system shall let students cancel a registration and let coordinators see participant lists.” | “ระบบต้องให้นักศึกษายกเลิกการลงทะเบียน และให้ผู้ประสานงานดูรายชื่อผู้เข้าร่วมได้” |

วิเคราะห์ประเภท จุดประสงค์ คุณภาพ วิธีตรวจ และสิ่งที่ควรปรับ โดยให้เหตุผลจากข้อความที่ได้รับ หากแยกเป็นหลายข้อ ต้องรักษาความต้องการเดิมให้ครบ และไม่เพิ่มสิทธิ์หรือเงื่อนไขใหม่โดยถือว่าเป็นข้อเท็จจริง

**คำตอบ**

**ข้อ 2 (7 คะแนน): ประเมินข้อเสนอโครงการ WorkshopHub**

| | |
|---|---|
| Students currently register for campus workshops through separate online forms. Coordinators combine the responses manually, sometimes causing duplicate registrations and delayed confirmations. | ปัจจุบันนักศึกษาสมัคร workshop ของมหาวิทยาลัยผ่านแบบฟอร์มออนไลน์ที่แยกจากกัน ผู้ประสานงานรวบรวมคำตอบด้วยมือ บางครั้งจึงเกิดการลงทะเบียนซ้ำและยืนยันล่าช้า |
| The team proposes WorkshopHub, a website for students and workshop coordinators. Students can browse and register for workshops, while coordinators manage participant lists. The team expects fewer errors and less administrative work. The first release supports free workshops only; payment and attendance certificates will come later. The team plans to finish before the next semester. | ทีมเสนอ WorkshopHub ซึ่งเป็นเว็บไซต์สำหรับนักศึกษาและผู้ประสานงาน นักศึกษาค้นหา workshop และลงทะเบียนได้ ส่วนผู้ประสานงานจัดการรายชื่อผู้เข้าร่วม ทีมคาดว่าจะลดข้อผิดพลาดและงานจัดการ รุ่นแรกครอบคลุม workshop ฟรีเท่านั้น การชำระเงินและใบรับรองจะทำภายหลัง ทีมวางแผนให้เสร็จก่อนภาคการศึกษาถัดไป |

**2.1 (4 คะแนน) ประเมินองค์ประกอบของข้อเสนอ**

ประเมินข้อเสนอตามที่เขียนจริงให้ครบ 8 องค์ประกอบ ระบุว่าแต่ละส่วนมีแล้ว มีเพียงบางส่วน หรือยังไม่มี พร้อมยกข้อความหรือเหตุผลประกอบ อย่านำข้อเสนอที่คุณคิดเพิ่มไปนับว่ามีอยู่ในต้นฉบับ

| Element<br>องค์ประกอบ | Status and evidence<br>สถานะและหลักฐาน |
|---|---|
| Problem | |
| Users | |
| Solution | |
| Expected value | |
| Scope | |
| Risk | |
| Effort | |
| Evidence of completion | |

**ข้อ 2 (ต่อ): ใช้ข้อเสนอ WorkshopHub จากหน้าก่อน**

**2.2 (3 คะแนน) เขียนข้อเสนอฉบับปรับปรุง**

เขียนข้อเสนอที่ช่วยให้ทีมตัดสินใจได้ว่าจะสร้างอะไร และช่วยให้ผู้ตรวจรับทราบว่างานเสร็จเมื่อใด คงปัญหา ผู้ใช้ และขอบเขตรุ่นแรกตามโจทย์ เติมหรือเสนอวิธียืนยันส่วนที่ยังขาด แยกข้อมูลเดิมออกจากคำแนะนำของคุณให้ชัด โดยไม่แต่งตัวเลขประมาณการขึ้นเป็นข้อเท็จจริง ตอบเป็นย่อหน้าต่อเนื่องหรือหัวข้อที่เชื่อมโยงกันได้

**คำตอบ**

**ข้อ 3 (4 คะแนน): สิทธิ์การใช้งานตามบทบาท**

| | |
|---|---|
| WorkshopHub assigns exactly one role to each account. Roles do not inherit permissions, and any operation not granted by the policy is denied. A Student may view published workshops and submit their own registrations. | WorkshopHub กำหนดหนึ่ง role ต่อหนึ่งบัญชี ไม่มีการสืบทอดสิทธิ์ระหว่าง roles และปฏิเสธ operation ที่นโยบายไม่ได้อนุญาต Student ดู workshop ที่เผยแพร่และส่งคำขอลงทะเบียนของตนเองได้ |
| A Coordinator may view published workshops, create workshops, edit workshop details, and export participant lists. No additional workshop-ownership restriction is specified. | Coordinator ดู workshop ที่เผยแพร่ สร้าง workshop แก้รายละเอียด workshop และส่งออกรายชื่อผู้เข้าร่วมได้ โจทย์ไม่ได้กำหนดข้อจำกัดตามเจ้าของ workshop เพิ่มเติม |

**3.1 (2.5 คะแนน) เติมตารางสิทธิ์ให้ครบทุกช่อง**

ใส่ Allow หรือ Deny ตามนโยบายข้างต้น ไม่สมมติว่าบทบาทหนึ่งได้รับสิทธิ์ของอีกบทบาทโดยอัตโนมัติ

| Operation (English) | รายการดำเนินการ | Student | Coordinator |
|---|---|---|---|
| View published workshops | ดู workshop ที่เผยแพร่ | | |
| Submit own registration | ส่งคำขอลงทะเบียนของตน | | |
| Create a workshop | สร้าง workshop | | |
| Edit workshop details | แก้ workshop | | |
| Export participant lists | ส่งออกรายชื่อ | | |

**3.2 (1.5 คะแนน) การซ่อนปุ่มเพียงพอหรือไม่**

| | |
|---|---|
| The team hides the export button from Students. | ทีมซ่อนปุ่มส่งออกรายชื่อจาก Student แล้ว |

การทำเช่นนี้เพียงพอหรือไม่? อธิบายสิ่งที่ backend ต้องตรวจ และเสนอ direct-request test หนึ่งกรณีพร้อมผลที่คาดหวัง

**คำตอบ**

**ข้อ 4 (6 คะแนน): วิเคราะห์ช่องโหว่ระบบส่งรายงาน**

| | |
|---|---|
| A university assignment system allows students to upload PDF reports but checks the file type only in the browser. A student bypasses this check and sends a script disguised as a PDF to the server. | ระบบส่งงานของมหาวิทยาลัยให้นักศึกษาอัปโหลดรายงาน PDF แต่ตรวจชนิดไฟล์เฉพาะใน browser นักศึกษาคนหนึ่งข้ามการตรวจดังกล่าว แล้วส่ง script ที่ปลอมเป็น PDF เข้าสู่ server |
| The server stores the file in a folder that permits script execution. Opening its URL causes the server to execute the script, which reads other students' reports. | Server เก็บไฟล์นั้นไว้ในโฟลเดอร์ที่อนุญาตให้รัน script เมื่อมีผู้เปิด URL ของไฟล์ server จึงรัน script และ script อ่านรายงานของนักศึกษาคนอื่นได้ |

ใช้เหตุการณ์ที่ระบุนี้ในการตอบ และแยกผลที่เกิดแล้วออกจากผลที่เพียงอาจเกิดขึ้น

**4.1 (3 คะแนน) วิเคราะห์เหตุการณ์และขอบเขตความเชื่อถือ**

ระบุผู้โจมตี สิ่งที่ต้องปกป้อง (assets) จุดที่ผู้โจมตีเข้าถึงระบบ (attack surface) และ trust boundary ที่ถูกข้าม อธิบาย security objective ที่เสียหายพร้อมเชื่อมกับข้อเท็จจริงในเหตุการณ์ อย่าตอบเพียงชื่อคำศัพท์โดยไม่มีเหตุผล

**คำตอบ**

**ข้อ 4 (ต่อ): ใช้เหตุการณ์อัปโหลดจากหน้าก่อน**

**4.2 (3 คะแนน) มาตรการป้องกันและการทดสอบเชิงลบ**

เสนอ control และ negative test จำนวน 2 คู่ แต่ละคู่ต้องระบุว่าจะป้องกันปัญหาตรงใด วิธีทดสอบที่พยายามข้ามการป้องกัน และผลที่ควรได้ เลือกมาตรการที่แก้ความเสี่ยงในเหตุการณ์จริง ไม่ใช้การตรวจใน browser เพียงอย่างเดียวเป็นมาตรการทั้งหมด

**คู่ที่ 1**

**คู่ที่ 2**

# ส่วนที่ 2 (25 คะแนน) การวางแผนและดำเนินโครงการ

**ข้อ 5 (10 คะแนน): วางแผนโครงการ DormFix**

| | |
|---|---|
| You are the project manager of DormFix, a Waterfall web application for dormitory repairs. Your team is Aom (BA), Bank (UX/UI), Chai (Developer) and Dao (Developer/Tester). The dormitory manager is the Sponsor, and the maintenance head represents the User. Tasks below are not in execution order. | คุณเป็นผู้จัดการโครงการ DormFix ซึ่งใช้ Waterfall เพื่อสร้างเว็บแจ้งซ่อมหอพัก ทีมมี Aom (BA), Bank (UX/UI), Chai (Developer), Dao (Developer/Tester) ผู้จัดการหอพักเป็น Sponsor และหัวหน้าช่างเป็นตัวแทน User งานในตารางไม่ได้เรียงตามลำดับ |

ให้ใช้ข้อมูลนี้ทำข้อ 5.1-5.3

| ID | Task (English) | งาน (ภาษาไทย) | Owner | Days |
|---|---|---|---|---|
| A | Code the request form from the approved designs | เขียนแบบฟอร์มแจ้งซ่อมจากแบบที่อนุมัติ | Chai | 4 |
| B | Submit the requirements for approval by the dormitory manager | ส่ง requirements ให้ผู้จัดการหอพักอนุมัติ | PM | 1 |
| C | Design the screens from the approved requirements | ออกแบบหน้าจอจาก requirements ที่อนุมัติ | Bank | 3 |
| D | Test the two coded parts together | ทดสอบโปรแกรมสองส่วนร่วมกัน | Dao | 2 |
| E | Write the requirements; this is the first task | เขียน requirements เป็นงานแรก | Aom | 3 |
| F | Submit both designs for approval by the maintenance head | ส่งแบบทั้งสองให้หัวหน้าช่างอนุมัติ | PM | 1 |
| G | Design the database from the approved requirements | ออกแบบฐานข้อมูลจาก requirements ที่อนุมัติ | Chai | 2 |
| H | Submit the tested system for acceptance by the dormitory manager | ส่งระบบที่ทดสอบแล้วให้ผู้จัดการหอพักตรวจรับ | PM | 1 |
| I | Code the technician dashboard from the approved designs | เขียน dashboard ช่างจากแบบที่อนุมัติ | Dao | 3 |

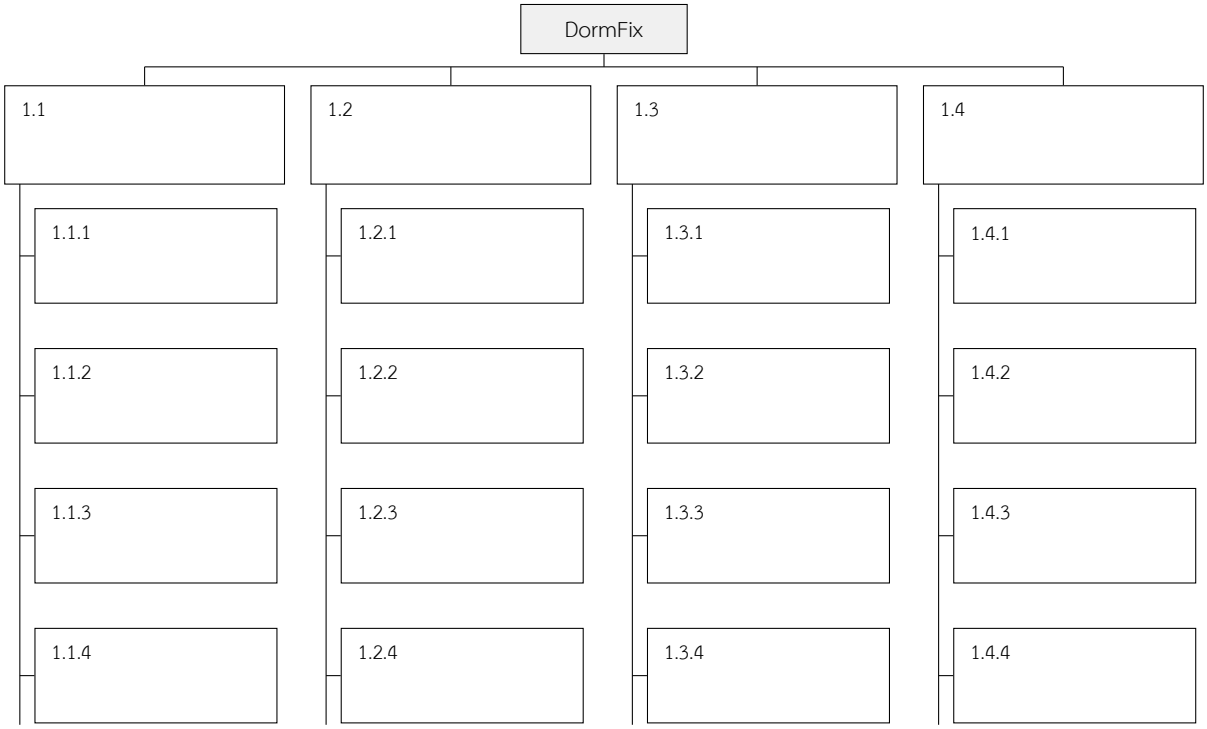
ใช้ชื่อผู้ทำและจำนวนวันตามตารางในการวางแผน รายละเอียดการอนุมัติและคำว่า “จากแบบที่อนุมัติ” หรือ “ระบบที่ทดสอบแล้ว” เป็นข้อมูลที่ต้องใช้กำหนดความสัมพันธ์ของงาน ไม่จำเป็นต้องใช้ทุกช่องของแบบฟอร์มคำตอบ

**ข้อ 5 (ต่อ): ใช้รายการงาน A-I**

**5.1 (3 คะแนน) จัด WBS ตาม milestone ที่บรรลุ**

เติมชื่อ milestone ระดับแรกในช่อง 1.1-1.4 เรียงจากซ้ายไปขวาตามลำดับที่บรรลุ แล้วใส่ตัวอักษรงาน A-I ลงในช่องงานระดับถัดไปตามลำดับที่ต้องทำเพื่อให้บรรลุ milestone นั้น

ใช้ทุกงานครั้งเดียว งานที่ทำขนานกันเรียงก่อนหลังในกลุ่มเดียวกันได้ ไม่เพิ่มงานหรือ milestone เพียงเพื่อเติมช่องให้เต็ม และเว้นช่องที่ไม่ใช้




พื้นที่ด้านล่างใช้ร่างหรืออธิบาย WBS ได้ คำตอบหลักต้องอยู่ในต้นไม้ข้างบน

**ข้อ 5 (ต่อ): บันทึกประชุมสำหรับ RACI**

**5.2 (2 คะแนน) เติม RACI ตามบทบาทที่ได้รับมอบหมาย**

พิจารณาบทบาทของแต่ละคนในแต่ละงานจากบันทึกประชุมด้านล่าง กรอก R, A, C, I หรือ “-” เมื่อไม่มีบทบาทให้ครบทุกช่อง ไม่อนุมานบทบาทเพิ่มจากตำแหน่งงานเพียงอย่างเดียว

| | |
|---|---|
| **E:** Aom performs the work; PM is accountable for the outcome; Bank is consulted beforehand; Chai is informed afterward; Dao has no part. | **E:** Aom ลงมือทำ PM รับผิดชอบผลสำเร็จ Bank ให้ความเห็นก่อนทำ Chai รับแจ้งหลังทำ Dao ไม่มีบทบาท |
| **C:** Bank performs the work; PM is accountable for the outcome; Aom is consulted beforehand; Chai is informed afterward; Dao has no part. | **C:** Bank ลงมือทำ PM รับผิดชอบผลสำเร็จ Aom ให้ความเห็นก่อนทำ Chai รับแจ้งหลังทำ Dao ไม่มีบทบาท |
| **G:** Chai performs the work; PM is accountable for the outcome; Bank is consulted beforehand; Dao is informed afterward; Aom has no part. | **G:** Chai ลงมือทำ PM รับผิดชอบผลสำเร็จ Bank ให้ความเห็นก่อนทำ Dao รับแจ้งหลังทำ Aom ไม่มีบทบาท |
| **D:** Dao performs the work; PM is accountable for the outcome; Chai is consulted beforehand; Aom is informed afterward; Bank has no part. | **D:** Dao ลงมือทำ PM รับผิดชอบผลสำเร็จ Chai ให้ความเห็นก่อนทำ Aom รับแจ้งหลังทำ Bank ไม่มีบทบาท |

| Task | PM | Aom | Bank | Chai | Dao |
|---|---|---|---|---|---|
| E | | | | | |
| C | | | | | |
| G | | | | | |
| D | | | | | |

**ข้อ 5 (ต่อ): เงื่อนไขการจัดตาราง**

**5.3 (5 คะแนน) วางกำหนดการเป็น Gantt chart**

**1 คะแนน (รวมอยู่ในข้อ 5.3):** อธิบายว่าคุณใส่ buffer ตรงใดและเพราะเหตุใด รวมทั้งเหตุผลที่ milestone ไม่ใช้เวลาทำงาน คำอธิบายต้องตรงกับตารางที่คุณวาด

| | |
|---|---|
| The project starts on Monday 4 January 2027. Work is Monday to Friday with no other holidays. Use the owner and duration in the A-I task table. A task starts on the working day after all its predecessors and their buffers have finished. | โครงการเริ่มวันจันทร์ที่ 4 มกราคม 2027 ทำงานวันจันทร์ถึงศุกร์และไม่มีวันหยุดอื่น ใช้ owner และจำนวนวันของแต่ละงานตามตาราง A-I งานหนึ่งเริ่มได้ในวันทำงานถัดจากวันที่ predecessors และ buffers ของงานเหล่านั้นเสร็จครบ |
| You and all four team members work full-time on DormFix. External stakeholders also have other duties. Add a 1-day task buffer and a 2-day project buffer only where necessary. | คุณและสมาชิกทีมทั้งสี่ทำ DormFix เต็มเวลา ส่วนผู้เกี่ยวข้องนอกทีมมีงานอื่นด้วย ใส่ task buffer 1 วันและ project buffer 2 วันเฉพาะจุดที่จำเป็น |
| A milestone is reached on the last marked day of its group's final task. The final milestone is reached on the last day of the project buffer. A milestone does not add a working day. | Milestone อยู่วันสุดท้ายที่ทำเครื่องหมายของงานสุดท้ายในกลุ่ม ส่วน milestone สุดท้ายอยู่วันสุดท้ายของ project buffer และไม่เพิ่มวันทำงานเพื่อทำ milestone |

ใช้ตารางหน้าถัดไป กรอก No. ตาม WBS, Task, Owner, Start และ Due โดย Start/Due เป็นวันแรก/สุดท้ายที่ทำเครื่องหมาย รวม buffer ใช้ X แทนวันทำงาน, - แทน buffer, * แทน milestone และแสดง dependency arrows เว้นแถวที่ไม่ใช้

**คำตอบ**

ชื่อ-นามสกุล _____ รหัสนักศึกษา _____

**ข้อ 5.3 (ต่อ): ตารางคำตอบ เดือนมกราคม 2027**

No. = รหัส WBS; Task = milestone หรือตัวอักษรงาน; Owner = ผู้ทำ (milestone ไม่มี owner); Start/Due รวม buffer; X = วันทำงาน, - = buffer, * = milestone; แสดงลูกศร dependency

| No. | Task | Owner | Start | Due | Week 1 (Jan) | | | | | Week 2 (Jan) | | | | | Week 3 (Jan) | | | | | Week 4 (Jan) | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | | | | M<br>4 | T<br>5 | W<br>6 | T<br>7 | F<br>8 | M<br>11 | T<br>12 | W<br>13 | T<br>14 | F<br>15 | M<br>18 | T<br>19 | W<br>20 | T<br>21 | F<br>22 | M<br>25 | T<br>26 | W<br>27 | T<br>28 | F<br>29 |
| | | | | | | | | | | | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | | | | | | | | | |
| | | | | | | | | | | | | | | | | | | | | | | | | |

**ข้อ 6 (10 คะแนน): Backlog grooming: จองสนามแบดมินตัน**

| | |
|---|---|
| You act as Product Owner while grooming Issue #12, “Book a badminton court.” The sports office, your client, wants students to book a court for one hour so they do not have to queue at the counter. | คุณทำหน้าที่ Product Owner ระหว่าง grooming ของ Issue #12 “Book a badminton court” ฝ่ายกีฬาซึ่งเป็น client ต้องการให้นักศึกษาจองสนามครั้งละ 1 ชั่วโมง เพื่อไม่ต้องต่อคิวที่เคาน์เตอร์ |
| Courts and opening hours are already stored in the system. A student may hold only one booking per day. Example: Court 2 is free at 17:00 today. After a student books it, Court 2 at 17:00 must be shown as “Booked.” | ระบบมีข้อมูลสนามและเวลาเปิดให้บริการแล้ว นักศึกษาแต่ละคนถือการจองได้ไม่เกิน 1 รายการต่อวัน ตัวอย่าง: สนาม 2 ว่าง เวลา 17:00 วันนี้ หลังนักศึกษาจองสำเร็จ สนาม 2 เวลา 17:00 ต้องแสดง “Booked” |
| The team has not agreed an assignee, milestone or estimate. | ทีมยังไม่ได้ตกลงผู้รับผิดชอบ milestone หรือ estimate |

ให้คงข้อมูลที่ได้รับ แยกข้อที่ต้องถามเพิ่มเติม และอย่าสมมติว่าทีมได้ตัดสินใจแล้ว

**6.1 (2 คะแนน) เขียน user story**

เขียน role, function และ value ให้เชื่อมกันตามความต้องการ client

| |
|---|
| |

**6.2 (2 คะแนน) ระบุ assumptions และ details**

บันทึกสิ่งที่ทีมทราบแล้วจาก client แยกประเด็นที่ยังต้องถามออกจากข้อเท็จจริง และไม่เปลี่ยนความหมายของข้อจำกัดหนึ่งการจองต่อวัน

| |
|---|
| |

**ข้อ 6 (ต่อ): เกณฑ์ยอมรับและ metadata ของ Issue #12**

**6.3 (4 คะแนน) เขียน Acceptance Criteria แบบ Gherkin**

เขียน 2 สถานการณ์พร้อม Given, When และ Then: (1) จองสำเร็จตามตัวอย่างสนาม 2 เวลา 17:00 วันนี้ และ (2) ขอจองเพิ่มเมื่อมีการจองในวันนั้นอยู่แล้ว แต่ละสถานการณ์ต้องมีเงื่อนไขตั้งต้น การกระทำ และผลที่ตรวจได้สอดคล้องกัน

**สถานการณ์ที่ 1**

| |
|---|
| |

**สถานการณ์ที่ 2**

| |
|---|
| |

**6.4 (2 คะแนน) กำหนด metadata หลัง grooming**

กรอกทั้งสี่ช่องตามสิ่งที่ตกลงแล้ว เลือก label ให้สอดคล้องกับประเภทงาน และใส่ “-” ในช่องที่ยังไม่มีค่า

| Assignees | Labels | Milestone | Estimate |
|---|---|---|---|
| | | | |

**ข้อ 7 (5 คะแนน): Sprint planning: ShuttleNow**

| | |
|---|---|
| You are Scrum Master of ShuttleNow, a campus shuttle tracking app. For Sprint 6, the Product Owner states the goal: “A student can track a shuttle and know when it arrives.” | คุณเป็น Scrum Master ของ ShuttleNow แอปติดตามรถรับส่งในมหาวิทยาลัย Product Owner ระบุเป้าหมาย Sprint 6 ว่า “นักศึกษาติดตามรถรับส่งและทราบเวลาที่รถจะมาถึงได้” |
| The team completed 16 story points last sprint and retains that velocity. The backlog is ranked by priority. The anchor, “View the list of shuttle routes,” is 3 points. Use the scale 1, 2, 3, 5, 8, 13, 21; a step larger or smaller means the next number on that scale. | ทีมทำได้ 16 story points ใน Sprint ก่อน และใช้ velocity เดิม Backlog เรียงตามความสำคัญ Anchor คือ “ดูรายการเส้นทางรถรับส่ง” = 3 points ใช้ scale 1, 2, 3, 5, 8, 13, 21 การใหญ่ขึ้นหรือเล็กลงหนึ่งขั้นหมายถึงเลขถัดไปบน scale |

**7.1 (3 คะแนน) ประเมิน points และเลือกงาน**

เติม points และ Yes/No ทุกแถว โดยใช้เป้าหมาย Sprint ข้อมูล capacity และลำดับความสำคัญที่ให้

| No. | User story | เรื่องผู้ใช้ | เทียบ anchor | Points | Yes/ No |
|---|---|---|---|---|---|
| 1 | See a shuttle's live position | ดูตำแหน่งรถสด | +2 steps | | |
| 2 | Pay the fare by QR code | จ่ายค่าโดยสาร QR | Same | | |
| 3 | See the arrival time at my stop | เวลาถึงป้ายฉัน | +1 step | | |
| 4 | Receive an alert before arrival | เตือนก่อนรถถึง | Same | | |
| 5 | See arrival times for all route stops | เวลาถึงทุกป้าย | +1 step | | |
| 6 | Fix the route list being cut off on small phones | แก้รายการเส้นทางถูกตัดบนมือถือ | -1 step | | |

**7.2 (1 คะแนน) รวม points ของงานที่เลือก**

| |
|---|
| |

**ข้อ 7 (ต่อ): อธิบาย Sprint backlog ที่เลือก**

**7.3 (1 คะแนน) อธิบายเหตุผลที่เลือกและไม่เลือก**

เชื่อมเหตุผลกับเป้าหมาย Sprint, velocity และลำดับงาน อธิบายให้ครบทั้งงานที่เลือกและไม่เลือก

# ส่วนที่ 3 (50 คะแนน) แผนภาพ UML การออกแบบหน้าจอ และ Git

**ข้อกำหนดร่วมสำหรับข้อ 8-10: ระบบแจ้งปัญหา TokTickIT**

ใช้ข้อกำหนด R1-R12 ของ TokTickIT ด้านล่างตอบข้อ 8-10 โดยยึดเฉพาะกฎที่ให้ ไม่ต้องเพิ่มทุกฟังก์ชันจากแลปจริง

| | |
|---|---|
| **R1. Accounts and role-based home pages.** After sign-in, route Requesters to the Requester Dashboard, IT Staff to the Staff Dashboard and Administrators to the Admin Dashboard. In the Use Case Diagram, User is the general actor; Requester and IT Staff specialize User, while Administrator specializes IT Staff for this model only. Requesters see only their own Tickets. Ticket Owner is the assigned IT Staff member, not a separate role. Administrators share Staff metrics and may add an account count. Enforce authorization on the server, even when controls are hidden. | **R1. บัญชีและหน้าแรกตามบทบาท** หลังเข้าสู่ระบบ ให้ Requester ไป Requester Dashboard, IT Staff ไป Staff Dashboard และ Administrator ไป Admin Dashboard ใน Use Case Diagram, User เป็น actor ทั่วไป; Requester และ IT Staff เป็นชนิดย่อยของ User ส่วน Administrator เป็นชนิดย่อยของ IT Staff เฉพาะในแบบจำลองนี้ Requester เห็นเฉพาะ Ticket ของตน Ticket Owner คือ IT Staff ที่ได้รับมอบหมาย ไม่ใช่ role แยก Administrator ใช้ metrics ของ Staff และอาจแสดงจำนวนบัญชีเพิ่ม Server ต้องตรวจสิทธิ์แม้ซ่อน control แล้ว |
| **R2. Create and track a Ticket.** A Requester creates a Ticket by entering its title, description and requested priority: LOW, MEDIUM or HIGH. On successful creation, the system assigns a ticket number, timestamps and the NEW status, then invokes Send Email through the Email Gateway. The Requester can view My Tickets and open details only for their own Tickets. | **R2. สร้างและติดตาม Ticket** Requester สร้าง Ticket โดยกรอก title, description และ requested priority ซึ่งเลือกได้เป็น LOW, MEDIUM หรือ HIGH เมื่อสร้างสำเร็จ ระบบกำหนด ticket number, เวลา และสถานะ NEW ให้อัตโนมัติ พร้อมเรียก Send Email ผ่าน Email Gateway ทุกครั้ง Requester ดูรายการ My Tickets และรายละเอียดได้เฉพาะ Ticket ของตนเอง |
| **R3. Ticket details and ownership.** Show the ticket number, title, description, requester name and email, both priorities, Owner, status and created/updated times. Account data, identifiers, timestamps and the Requester's original text are read-only. A Ticket may be unassigned or have one active IT Staff member as Owner. IT Staff may edit IT priority, Owner and status as permitted. | **R3. ข้อมูลและผู้รับผิดชอบ Ticket** หน้ารายละเอียดแสดง ticket number, title, description, ชื่อและอีเมล requester, requested priority, IT priority, Owner, status และ created/updated time ข้อมูลบัญชี หมายเลข วันเวลา และข้อความต้นฉบับของ Requester แก้ไขไม่ได้ Ticket อาจไม่มี Owner หรือมี IT Staff ที่ยัง active เป็นผู้รับผิดชอบได้หนึ่งคน IT Staff แก้ IT priority, Owner และ status ได้ตามสิทธิ์ |
| **R4. Staff Dashboard and Ticket Queue.** Show unassigned Tickets, the current user's Tickets, counts by status and IT priority, and recent updates. Each metric opens its Queue. Search by title, filter by status, IT priority or Owner, and paginate. Rows show the ticket number, title, requester, status, both priorities, Owner and updated time; selecting one opens Form View. Distinguish an empty system from no matches. | **R4. Staff Dashboard และ Ticket Queue** Staff Dashboard แสดง Ticket ที่ไม่มี Owner, Ticket ของผู้ใช้ปัจจุบัน, จำนวนแยกตาม status/IT priority และรายการล่าสุด แต่ละ metric เปิด Queue ที่เกี่ยวข้องได้ Queue ค้น title, กรอง status/IT priority/Owner และเปลี่ยนหน้าได้ แถวแสดง ticket number, title, requester, status, requested/IT priority, Owner และ updated time; เลือกแล้วเปิด Form View ต้องแยก empty state จากกรณีค้นหาไม่พบ |
| **R5. Updates.** Update Ticket changes an editable IT priority or status. Reassign Ticket is optional and occurs only when IT Staff choose another active Owner. Other updates do not change ownership. | **R5. การปรับ Ticket** Update Ticket ใช้เปลี่ยน IT priority หรือ status ที่มีสิทธิ์แก้ ส่วน Reassign Ticket เป็นพฤติกรรมเสริมเมื่อ IT Staff เลือก Owner คนใหม่ที่ยัง active; การอัปเดตอื่นไม่ต้องเปลี่ยน Owner |

| | |
|---|---|
| **R6. Actions and files.** Record Action stores work text and an OPEN or COMPLETED result, always invoking Send Email on success. Attach File optionally extends it when a file is selected, and always invokes Validate Attachment for type/size checks. An Action need not have a file. | **R6. Action และไฟล์** IT Staff ใช้ Record Action บันทึกข้อความและผล OPEN หรือ COMPLETED โดยต้อง Send Email ทุกครั้งที่บันทึกสำเร็จ Attach File เป็นพฤติกรรมเสริม เฉพาะเมื่อเลือกไฟล์ และต้อง Validate Attachment เพื่อตรวจชนิด/ขนาดเสมอ บันทึก Action โดยไม่แนบไฟล์ได้ |
| **R7. Communication.** A Ticket's Requester and IT Staff can read/write Public Comments. Only IT Staff and Administrators can read/write Internal Notes. Email recipients must be authorized to see its contents. A Requester sees neither Internal Notes nor their count. | **R7. การสื่อสาร** Public Comments ให้ Requester ของ Ticket และ IT Staff อ่าน/เขียนได้ ส่วน Internal Notes ให้เฉพาะ IT Staff และ Administrator อ่าน/เขียน ผู้รับ Email ต้องมีสิทธิ์เห็นข้อความนั้นเสมอ Requester ไม่เห็น Internal Notes หรือจำนวนรายการภายใน |
| **R8. Resolution.** Resolve Ticket always invokes Check Resolution Gate. An owner, at least one COMPLETED Action, and no OPEN Actions are required. If any condition fails, refuse the operation, explain why, and retain the current status. | **R8. การแก้เสร็จ** Resolve Ticket ต้องเรียก Check Resolution Gate ทุกครั้ง เงื่อนไขคือมี owner มี Action ที่ COMPLETED อย่างน้อยหนึ่งรายการ และไม่มี Action ที่ยัง OPEN ถ้าขาดข้อใดให้ปฏิเสธ แสดงเหตุผล และคงสถานะเดิม |
| **R9. Start and pause work.** A created Ticket is NEW. Staff claim it to OPEN, start it to IN_PROGRESS, request information to WAITING_FOR_REQUESTER, and resume it to IN_PROGRESS after receiving the information. Posting a Comment alone does not change status. | **R9. เริ่มและพักงาน** Ticket ใหม่เป็น NEW; IT Staff claim แล้วเป็น OPEN; เริ่มทำงานจาก OPEN เป็น IN_PROGRESS; ขอข้อมูลจาก IN_PROGRESS เป็น WAITING_FOR_REQUESTER; เมื่อได้รับข้อมูลแล้ว IT Staff resume กลับเป็น IN_PROGRESS การเขียน Comment อย่างเดียวไม่เปลี่ยนสถานะ |
| **R10. Finish or cancel.** IT Staff resolve IN_PROGRESS to RESOLVED only after R8 passes. The Requester either accepts to CLOSED or reopens to IN_PROGRESS. They can cancel their own NEW or OPEN Ticket to CANCELLED only. CLOSED and CANCELLED are terminal. No other transitions are permitted. | **R10. จบและยกเลิก** IT Staff resolve จาก IN_PROGRESS เป็น RESOLVED เมื่อผ่าน R8; Requester ยอมรับผลเป็น CLOSED หรือ reopen เป็น IN_PROGRESS ได้ Requester ยกเลิก Ticket ของตนจาก NEW หรือ OPEN เป็น CANCELLED เท่านั้น CLOSED และ CANCELLED เป็น terminal states ไม่อนุญาต transition อื่นนอกเหนือจากที่ระบุ |
| **R11. Validation and stale data.** Blank text cannot be saved. If the submitted version is stale, preserve the user's input, show the current record and offer Refresh without automatic resubmission. Show field errors or refusal reasons near the relevant control. | **R11. Validation และ stale data** ข้อความว่างบันทึกไม่ได้ หาก version ที่ส่งเก่าให้คงค่าที่ผู้ใช้พิมพ์ แสดงสถานะล่าสุด และให้ Refresh โดยไม่ส่งซ้ำอัตโนมัติ แสดง field errors หรือเหตุผลที่บันทึกไม่ได้ใกล้ control ที่เกี่ยวข้อง |
| **R12. Diagram scope.** In Question 8 add relationships among the supplied actors and use cases, including Sign In, role-specific Dashboards and the stated inheritance. In Question 9 connect the supplied states. In Question 10 complete the three role dashboards and the Queue/Detail screens in the provided frames. Do not add account registration, payment or other external services. | **R12. ขอบเขตภาพ** ข้อ 8 เติม relationships ของ actors/use cases ที่ให้ รวม Sign In, หน้า Dashboard ตาม role และ inheritance ที่ระบุ ข้อ 9 เติม transitions ของ states ที่ให้ ข้อ 10 เติม Dashboard ของทั้งสาม role และหน้าจอ Queue/Detail ในกรอบ ไม่ต้องสร้างบัญชีใหม่ ระบบชำระเงินหรือบริการภายนอกอื่น |

**ข้อ 8 (10 คะแนน): เติมความสัมพันธ์ใน Use Case Diagram**

**เกณฑ์คะแนน:** associations, การกำหนด Dashboard ตาม role และขอบเขต 3 คะแนน; include 3 คะแนน; extend พร้อมเงื่อนไข 2 คะแนน; generalization และทิศทาง 2 คะแนน รวม 10 คะแนน

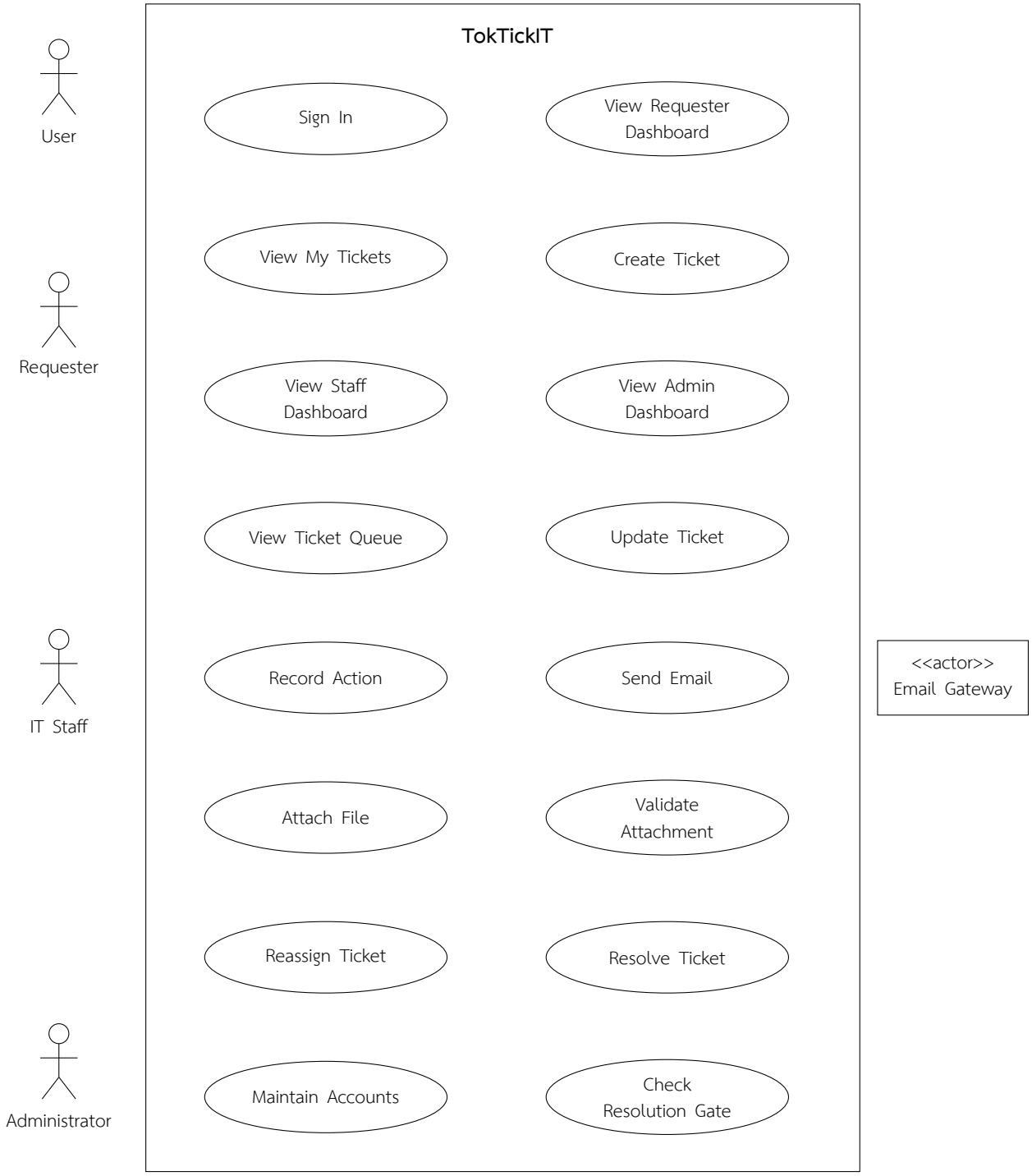
ใช้ข้อกำหนด R1-R8 และ R12 เติมความสัมพันธ์ในโครงหน้าถัดไป ซึ่งมี actors, use cases และ system boundary ให้แล้ว ห้ามเพิ่ม node ใช้เส้นทึบแทน association และเส้นประพร้อมป้าย <<include>> หรือ <<extend>> สำหรับ dependency ส่วน generalization ใช้เส้นทึบหัวสามเหลี่ยมโปร่ง โดยให้หัวชี้ไปยัง actor ทั่วไป

เชื่อม User กับ Sign In; Requester กับ Requester Dashboard, Create Ticket และ View My Tickets; IT Staff กับ Staff Dashboard, View Ticket Queue, Update Ticket, Record Action และ Resolve Ticket; Administrator กับ Admin Dashboard และ Maintain Accounts ไม่ต้องวาด association ซ้ำสำหรับความสามารถที่สืบทอดมา เชื่อม Email Gateway กับ use case ที่เรียกใช้บริการนี้

แยกพฤติกรรมที่ต้องเกิดทุกครั้งออกจากพฤติกรรมที่เกิดเมื่อเลือกแนบไฟล์หรือเปลี่ยน Owner พร้อมระบุทิศลูกศร stereotype และเงื่อนไขของ extension ให้ถูกต้อง แผนภาพนี้แสดงขอบเขตความสามารถ ไม่ใช่ลำดับการทำงานหรือการเปลี่ยน status

**คำอธิบายหรือสมมติฐาน**

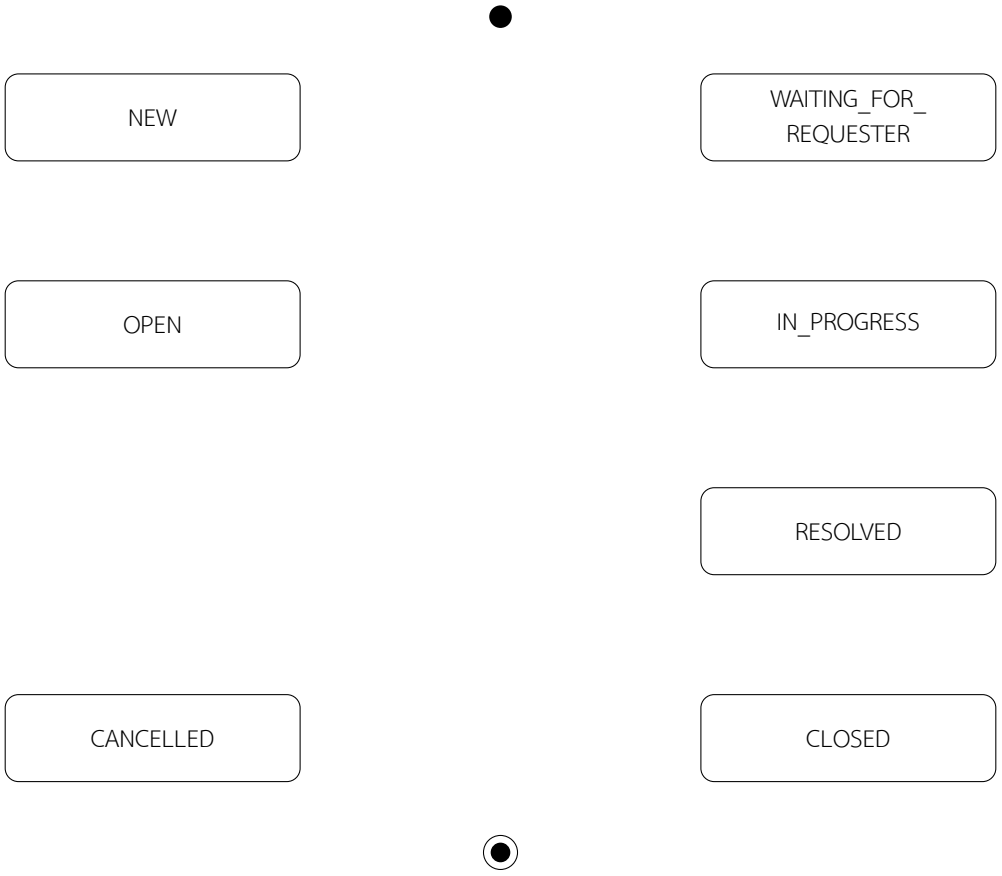
**ข้อ 8 (ต่อ): เติมเส้นและป้ายความสัมพันธ์ในโครงที่ให้**



**ข้อ 9 (10 คะแนน): เติม State Diagram ของ Ticket**

ใช้ R8-R10 เชื่อม states ที่ให้ด้วยลูกศรแบบมีทิศทาง ระบุ event, ผู้กระทำ และ guard ที่จำเป็น แสดงจุดเริ่มและทางออกของ terminal states อย่าลากเส้น include, extend หรือ actor generalization ในภาพนี้ เพราะเป็น State Diagram

Ticket Owner คือ IT Staff ที่ถูกกำหนดให้ Ticket นั้น ไม่ใช่ role ใหม่ ต้องแยก claim, start, request information, resume, resolve, accept, reopen และ cancel ให้ครบ claim กำหนด Owner; ใส่เงื่อนไข Resolve ของ R8 บนลูกศรหรือใช้ชื่อ guard แล้วอธิบายด้านล่าง การกระทำที่ไม่ผ่าน guard คงสถานะเดิม ไม่ต้องเพิ่มสถานะ Error




**นิยาม guard**

| |
|---|
| |

**ข้อ 10 (10 คะแนน): เติม Dashboard ตาม role และหน้าจอ Ticket**

ใช้ข้อกำหนด R1-R4 และ R7-R12 เติม Dashboard ของ Requester, IT Staff และ Administrator ลงในกรอบที่ให้ หลัง sign in ผู้ใช้แต่ละบทบาทต้องเข้าสู่หน้าแรกที่ตรงกับบทบาทของตน ไม่ต้องวาดหน้า Login จากนั้นเติมหน้าจอ Queue และ Detail ด้วย ใช้ * ระบุ required fields, ใช้ textbox แทนข้อมูลที่แก้ไขได้ และใช้เส้นเฉียงแสดงข้อมูล read-only หรือ calculated

| *Required field | Editable | Read-only | LoV / Selection |
|---|---|---|---|

ตัวอย่างสัญลักษณ์เท่านั้น ให้เติมข้อมูลลงในกรอบว่างด้านล่าง

**10.1 (3 คะแนน) หน้าแรกและ Dashboard ตาม role**

เติมข้อมูลและเส้นทางไปยังรายการที่เกี่ยวข้องให้ครบทั้งสาม Dashboard Requester เห็นเฉพาะ Ticket ของตนที่ยังเปิด รอข้อมูลเพิ่งอัปเดต หรือเพิ่งแก้ไขเสร็จ IT Staff เห็น Ticket ที่ยังไม่มี Owner, Ticket ที่ตนดูแล, จำนวนแยกตาม status หรือ IT priority และรายการที่เพิ่งอัปเดต Administrator ใช้ metrics ของ Staff และอาจเพิ่มจำนวนบัญชีได้ เมื่อกด metric ต้องเปิดรายการที่เกี่ยวข้อง

**Requester Dashboard**

Summary cards / metrics

| Metric 1 | Metric 2 | Metric 3 |
|---|---|---|

Recent / needs-attention items, navigation and drill-down

**IT Staff Dashboard**

Summary cards / metrics

| Metric 1 | Metric 2 | Metric 3 |
|---|---|---|

Recent / needs-attention items, navigation and drill-down

**Administrator Dashboard**

Summary cards / metrics

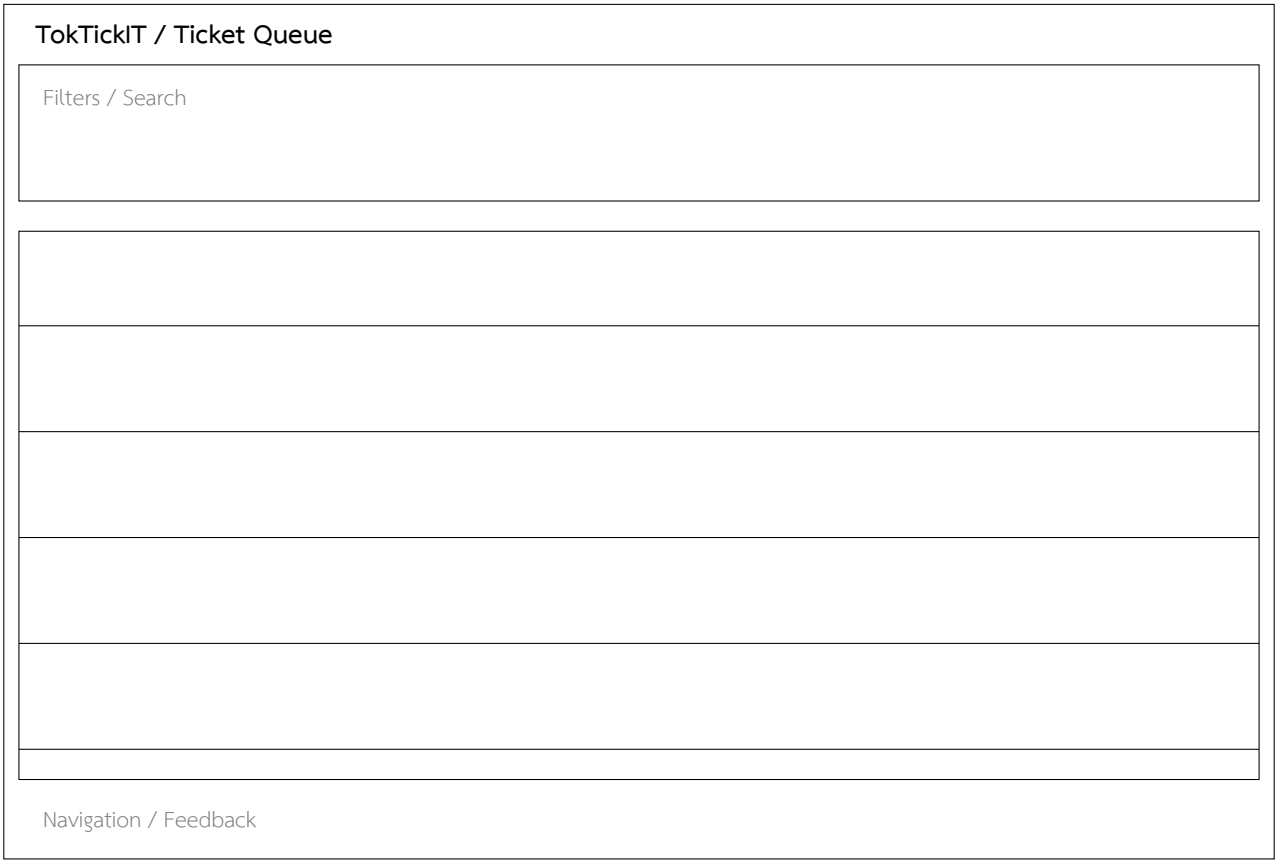
| Metric 1 | Metric 2 | Metric 3 |
|---|---|---|

Recent / needs-attention items, navigation and drill-down

**ข้อ 10 (ต่อ): Ticket Queue ของ IT Staff**

**10.2 (3 คะแนน) ค้นหาและกรอง Ticket ตาม Owner**

แสดงข้อมูลทุกช่องที่ระบุใน R4 เพิ่มช่องค้นหาจาก title และตัวกรอง status, IT priority และ Owner ให้เลือกแถวเพื่อเปิด Ticket และเปลี่ยนหน้าได้ แสดงข้อความแยกกันเมื่อระบบยังไม่มี Ticket กับเมื่อไม่มีผลลัพธ์จากตัวกรอง และมีทางล้างตัวกรอง



**ข้อ 10 (ต่อ): Staff Ticket Detail**

**10.3 (4 คะแนน) รายละเอียด Ticket ผู้รับผิดชอบ และการดำเนินงาน**

เติมข้อมูลตาม R3 โดยแยกค่าที่แก้ไขไม่ได้ออกจาก IT priority, Owner และ status ที่ IT Staff เปลี่ยนได้ รายชื่อ Owner ต้องมีเฉพาะ IT Staff ที่ยัง active แยก Public Comments ออกจาก Internal Notes และมีช่องบันทึก Action พร้อมแนบไฟล์ได้ เมื่อ Resolve ไม่ผ่านเงื่อนไข, ข้อความว่าง หรือเกิด STALE_UPDATE ให้แจ้งสาเหตุ คงข้อมูลที่ผู้ใช้กรอกไว้ และไม่ส่งคำขอซ้ำโดยอัตโนมัติ

**TokTickIT / Selected Ticket**

Ticket information / Controls

Actions / Threads

Messages / Allowed actions

**บริบทร่วมสำหรับข้อ 11-12: Git Workflow**

| | |
|---|---|
| ExamScore is a command-line program instructors use after an examination. It reads names and marks from a CSV file, checks that each mark is between 0 and 100, calculates summary statistics and produces a pass-rate report. Python keeps the code and CSV example straightforward so the exercise can focus on Git. | ExamScore เป็นโปรแกรม command-line ที่อาจารย์ใช้หลังสอบเพื่ออ่านรายชื่อและคะแนนจากไฟล์ CSV ตรวจว่าคะแนนอยู่ระหว่าง 0-100 คำนวณสถิติ และจัดทำรายงานอัตราผ่าน โจทย์ใช้ Python เพื่อให้ตัวอย่างโค้ดและการจัดการ CSV เข้าใจง่าย โดยเน้นการฝึก Git |
| In this exercise, main represents the deployed release. Sprint branches collect work for testing, while feature branches isolate individual changes before integration. | ในโจทย์นี้ main แทนรุ่นที่นำขึ้นใช้งานแล้ว ส่วน sprint branches ใช้รวบรวมงานเพื่อทดสอบ และ feature branches ใช้แยกงานแต่ละส่วนก่อนนำไปรวม |

| File | Purpose (English) | หน้าที่ (ภาษาไทย) |
|---|---|---|
| **score_data.py** | Loads and validates exam scores | โหลดและตรวจสอบคะแนนสอบ |
| **scores.csv** | Sample student names and scores | รายชื่อและคะแนนตัวอย่าง |
| **validation_notes.txt** | New file documenting validation cases | ไฟล์ใหม่บันทึกกรณีทดสอบ |

| | |
|---|---|
| HEAD points to the checked-out branch. The origin/* names are remote-tracking references that record the latest remote state known to this machine. Switching changes HEAD but does not move other branches. | HEAD ชี้ไปยัง branch ที่กำลัง checkout. origin/* เป็น remote-tracking references ซึ่งบันทึกสถานะล่าสุดของ remote ที่เครื่องนี้ทราบ. การ switch เปลี่ยน HEAD แต่ไม่ย้าย branch อื่น |
| Assume a clean working tree, successful commands and no conflicts unless stated otherwise. If no fetch, push or other remote command appears, assume GitHub is not contacted. Each question starts from its own graph. | ให้ถือว่า working tree สะอาด คำสั่งทำงานสำเร็จ และไม่มี conflict เว้นแต่โจทย์จะระบุไว้ หากไม่มีคำสั่ง fetch, push หรือคำสั่ง remote อื่น ให้ถือว่าไม่ได้ติดต่อ GitHub แต่ละข้อเริ่มจาก graph ของตนเอง |
| In a graph, A---B means commit B has A as its parent. Bracketed names and ref -> commit identify reference locations; HEAD -> branch marks the current branch. The / and \ lines show ancestry. A leading > marks a command. Answers may use ASCII or a drawn graph. | ในกราฟ A---B หมายถึง commit B มี parent คือ A. ชื่อในวงเล็บเหลี่ยมหรือรูปแบบ ref -> commit ใช้ระบุตำแหน่งของ ref; HEAD -> branch ระบุ branch ปัจจุบัน. เครื่องหมาย / และ \ แสดงเส้นทาง ancestry ส่วน > ใช้นำหน้าคำสั่ง ผู้สอบตอบด้วยกราฟตัวอักษรหรือวาดแผนภาพได้ |

**ใช้ชื่อไฟล์เมื่อ stage ห้ามใช้ git add .** แสดง local branches, origin/* และ HEAD ตามคำสั่งของแต่ละขั้น อย่านำ graph สุดท้ายของข้อ 11 ไปเริ่มข้อ 12

**ข้อ 11 (10 คะแนน): แก้ข้อผิดพลาดและรวมงานตรวจคะแนนเข้า sprint_1**

| | |
|---|---|
| The team is preparing an ExamScore test release. The program validates marks from CSV files and produces reports. The deployed version is at C on **main**. The Sprint team branched **sprint_1** from B and created D to revise the report. Another team branched **feature/load_scores** from B and created E to improve score loading. Both changes are needed in the test release. | ทีมกำลังเตรียมรุ่นทดสอบของ ExamScore โปรแกรมที่ใช้ตรวจคะแนนจาก CSV และจัดทำรายงาน รุ่นที่นำขึ้นใช้งานแล้วอยู่ที่ commit C บน **main** ทีม Sprint แยก **sprint_1** จาก B และสร้าง D เพื่อปรับรูปแบบรายงาน ส่วนอีกทีมแยก **feature/load_scores** จาก B และสร้าง E เพื่อปรับการอ่านคะแนน ทั้งสองงานต้องอยู่ในรุ่นทดสอบ |
| During review, the team finds that **score_data.py** still accepts scores below 0 or above 100. A developer has fixed it on **feature/load_scores**, but has not staged or committed the change. This is the only modified file; **scores.csv** is unchanged. | ระหว่างตรวจรุ่นทดสอบ พบว่า **score_data.py** ยังรับคะแนนต่ำกว่า 0 หรือสูงกว่า 100 ได้ ผู้พัฒนาแก้ข้อผิดพลาดบน **feature/load_scores** แล้ว แต่ยังไม่ได้ stage หรือ commit และไฟล์นี้เป็นรายการเดียวที่แก้ใน working tree ส่วน **scores.csv** ไม่เปลี่ยน |

ให้ทำตามคำสั่งด้านล่างเพื่อบันทึก F รวม feature เข้า **sprint_1** โดยเก็บ D ไว้ แล้วสลับกลับ **main** โดยไม่ push

```
A---B---C [main, origin/main]
     |\
     D E [HEAD -> feature/load_scores, origin/feature/load_scores]
sprint_1, origin/sprint_1 -> D
```

working tree มีเฉพาะการแก้ **score_data.py** ที่ระบุไว้ ให้ใช้คำสั่งต่อไปนี้ตามลำดับ สมมติว่าไม่มี conflict และเมื่อรวม branch ที่แยกจากกัน Git จะสร้าง merge commit

```
> git status
> git diff -- score_data.py
> git add -- score_data.py
> git commit -m "Reject out-of-range scores"
> git switch sprint_1
> git merge feature/load_scores
> git switch main
```

**11.1 (2 คะแนน) วาด commit graph หลังสร้าง F**

แสดง local branches, origin/* และ HEAD โดยไม่ย้าย refs ที่คำสั่งยังไม่เปลี่ยน

**ข้อ 11 (ต่อ): รวม feature เข้า sprint_1 และกลับไป main**

**11.2 (5 คะแนน) วาด graph หลัง merge เข้า sprint_1**

วาด commit graph พร้อม refs ทั้งหมด ระบุชนิดของ merge, ancestor ร่วม และ parent ทั้งสองของ merge commit G. อธิบายว่า local branch ใดเลื่อน และ origin/* ใดคงเดิม

**11.3 (3 คะแนน) วาด graph หลัง switch กลับไป main**

หลังคำสั่งสุดท้าย HEAD และ branch/ref แต่ละตัวชี้อยู่ที่ใด อธิบายด้วยว่า switch สร้าง commit ใหม่หรือไม่

**ข้อ 12 (10 คะแนน): เขียนคำสั่งและวาด graph ทีละขั้น**

| | |
|---|---|
| Question 12 starts afresh: the feature was branched at C and has since added D and E in sequence. HEAD is on feature/load_scores. | ข้อ 12 เริ่มใหม่ตามกราฟนี้: feature แยกจาก C และได้ทำ commit D กับ E ต่อกันไว้แล้ว ขณะนี้ HEAD อยู่ feature/load_scores |

ทำ 12.1-12.3 ต่อเนื่องโดยคง refs ที่ไม่มีคำสั่งเปลี่ยน ไม่ใช้ผลจากข้อ 11

```
A---B [main, origin/main]
     \
      C [sprint_1_staging, origin/sprint_1_staging]
       \
        D---E [HEAD -> feature/load_scores]
```

**12.1 (2 คะแนน) เปลี่ยนไป sprint staging branch**

เขียนคำสั่งเปลี่ยนจาก feature/load_scores ไป sprint_1_staging แล้ววาด graph ใหม่ให้แสดงตำแหน่ง HEAD และ branch/remote-tracking references หลังคำสั่ง

**คำสั่ง**

**Graph หลังข้อ 12.1**

**ข้อ 12 (ต่อ): เริ่มจากผลของข้อ 12.1**

**12.2 (3 คะแนน) รวม feature branch เข้า staging**

ขณะนี้อยู่ที่ sprint_1_staging ให้เขียนคำสั่งรวม feature/load_scores เข้าสู่ branch ปัจจุบัน ระบุ merge type พร้อมเหตุผลจาก ancestry แล้ววาด graph หลัง merge โดยแสดง HEAD, local branches และ remote-tracking references ทุกตัว

**คำสั่งและประเภท merge**

**Graph หลังข้อ 12.2**

**ข้อ 12 (ต่อ): เริ่มจากผลของข้อ 12.2**

**12.3 (5 คะแนน) ตรวจการเปลี่ยนแปลง stage และ commit**

| | |
|---|---|
| After 12.2, the developer uses an editor to modify **score_data.py** and **scores.csv** and create **validation_notes.txt**. | หลังข้อ 12.2 ผู้พัฒนาใช้ editor แก้ **score_data.py** และ **scores.csv** และสร้างไฟล์ใหม่ **validation_notes.txt** |

การแก้เนื้อหาเกิดขึ้นแล้ว คุณไม่ต้องเขียนโค้ด Python หรือคำสั่ง editor

เขียนคำสั่ง Git เพื่อตรวจการเปลี่ยนแปลง stage เฉพาะสามไฟล์นี้ และสร้าง commit F ด้วยข้อความ **Document score validation cases** จากนั้นวาด graph โดยแสดง F, HEAD และ references ทุกตัว ไม่มีคำสั่ง push ในขั้นนี้

**คำสั่ง**

**Graph หลังสร้าง F**